$ whoami

Zakaria Ech.Chifaouy

Software engineer from the 42 Network (1337) working across **AI engineering**, **systems & web**, and **security**. Former AI Engineering intern at Oracle R&D Morocco.

EMAIL
zech.chi.work@gmail.com

GITHUB
github.com/zech-chi

LINKEDIN
in/zakaria-ech-chifaouy

PHONE
+212 601 411 639

About me

I’m a software engineer who wants to understand systems all the way down — from processes and sockets to LLM agents — and turn that understanding into software that works.

I learned to work that way at 1337 (42 Network), where there are no lectures: projects are built in teams, reviewed by peers, and defended in depth. Before that, two years of integrated preparatory classes at ENSAM Casablanca gave me a solid base in mathematics and engineering sciences.

In 2026 I spent six months as a Research Assistant at Oracle R&D Morocco, building an agent-based Text-to-SQL pipeline and benchmarking it on BIRD and Spider. I'm also a staff member of the 1337 Competitive Programming Club.

EDUCATION

2023 – now

1337 Coding School

42 Network & UM6P · Khouribga

Software Engineering — common core completed, now in the advanced track.

2020 – 2022

ENSAM Casablanca

Integrated preparatory classes — mathematics, physics, engineering sciences.

2019 – 2020

Lycée Allal Ben Abdellah

Baccalauréat International, Sciences Mathématiques A.

LANGUAGES

Arabic native

English fluent

French intermediate

6 mo
AI engineering at Oracle R&D Morocco

42
Network curriculum — common core completed

2024→
Staff, 1337 Competitive Programming Club

10
Projects in this portfolio, from C to AI

Toolbox

Programming

C · C++ · Python · JavaScript · TypeScript · SQL · Bash

Web & databases

Next.js · Node.js · Fastify · Socket.IO · REST APIs · SQLite · MariaDB · Redis

AI & data

LangGraph · Text-to-SQL · Tool-calling agents · pandas · NumPy · scikit-learn · Matplotlib

Systems

Linux · POSIX processes · pthreads · Semaphores · Sockets · kqueue

DevOps & tools

Git · Docker · Docker Compose · Nginx · Prometheus · Grafana

Security

Web security · OWASP Top 10 · Vulnerability assessment

Selected work

01	**Text-to-SQL agent** Oracle R&D	AI
02	**ft_transcendence** Full-stack multiplayer platform	WEB
03	**Inception** Containerised WordPress infra	DEVOPS
04	**WebServ** HTTP/1.1 server in C++	SYSTEMS
05	**Minishell** A bash-like shell in C	SYSTEMS
06	**Philosophers** Concurrency: threads & processes	SYSTEMS
07	**Darkly** Web penetration testing	SECURITY
08	**NFL analytics** Data analysis & ML	DATA
09	**cub3D** Raycasting 3D engine in C	GRAPHICS
10	**so_long** 2D tile game in C	GRAPHICS

01 — Experience · AI

Text-to-SQL agent

Research Assistant — AI Engineering Intern · Oracle R&D Morocco · Feb – Aug 2026 · Casablanca

Python · LangGraph · Tool-calling · LLMs · SQL · BIRD · Spider

THE PROBLEM

Oracle customers store critical business data in databases, but getting answers out of it usually means writing SQL. The goal: let users ask everyday questions in natural language and get correct results back.

WHAT I BUILT

A Text-to-SQL (NL2SQL) pipeline written from scratch in Python — schema handling, prompt construction, query generation and execution — then extended into an agent that inspects the database itself and fixes its own mistakes.

ENGINEERING HIGHLIGHTS

- **Pipeline from scratch:** schema extraction and formatting, prompt design, SQL generation and execution.
- **Agentic workflow:** LangGraph + tool-calling — the agent inspects database structure, runs queries, and self-corrects on errors.
- **Evaluation:** benchmarked the original pipeline against the agent-based workflow on BIRD and Spider by comparing query results.



02 — Web

ft_transcendence

Team project · “Game Zone” — play together, chat with friends, track your progress

TypeScript · Next.js · Fastify · Socket.IO · SQLite · Babylon.js · Docker

THE PROBLEM

The final project of the 42 common core: a web platform where users play together in real time and stay in touch — built by a team, under strict constraints on stack and security.

WHAT I BUILT

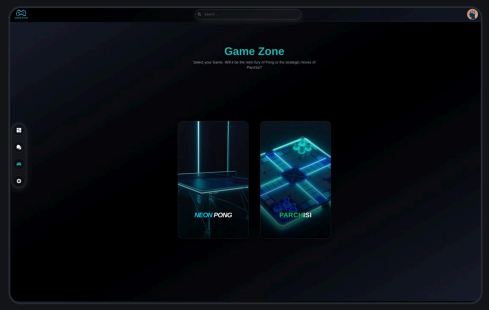
Game Zone: Pong (local, online, vs friends, tournaments) and a 3D Parcheesi in Babylon.js, plus live chat, friends, profiles and a stats dashboard — on a backend split into small services.

ENGINEERING HIGHLIGHTS

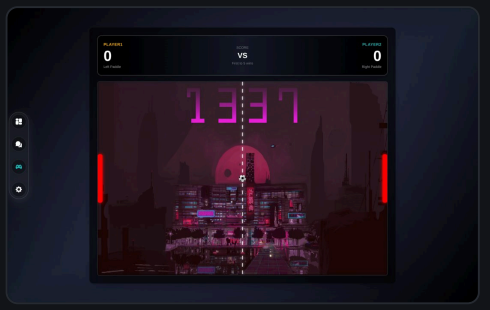
- **Real-time:** Socket.IO for gameplay, chat, file & voice messages; audio/video calls over WebRTC.
- **Accounts:** email verification, Google sign-in, and two-factor authentication.
- **Services:** Fastify microservices (accounts, chat, games) on SQLite, run with Docker Compose.
- **Observability:** Prometheus + Grafana metrics and ELK-based log search in Kibana.

github.com/zech-chi/ft_trancendance_42_network ↗

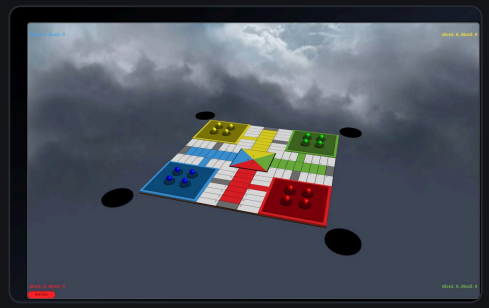
SCREENSHOTS



Game selection



Pong match



3D Parcheesi



Dashboard

03 — DevOps

Inception

System administration · eight-service containerised infrastructure

Docker · Docker Compose · Nginx · MariaDB · WordPress · Redis · TLS

THE PROBLEM

Deploy a production-like WordPress site where every service is isolated, connections are protected, and data survives container restarts — with every image built from a custom Dockerfile.

WHAT I BUILT

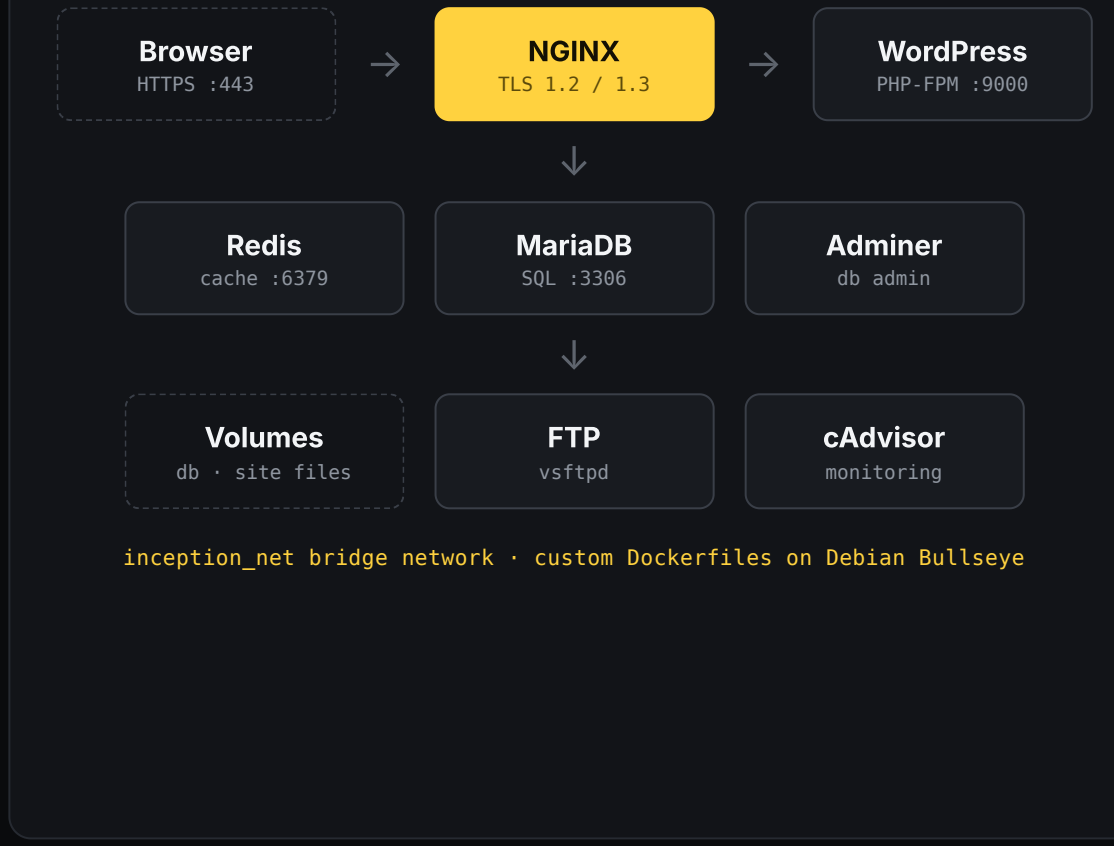
A Docker Compose stack of eight services built on Debian Bullseye: the WordPress core plus Redis caching, FTP, Adminer, a static website and cAdvisor monitoring.

ENGINEERING HIGHLIGHTS

- **Single entry point:** NGINX on :443 with TLSv1.2/1.3, forwarding PHP to WordPress over FastCGI.
- **Isolation:** WordPress (PHP-FPM) and MariaDB are internal-only on a private bridge network.
- **Persistence:** volumes for the database and WordPress files survive restarts.
- **Secrets:** credentials kept out of images in a dedicated secrets folder.
- **Bonus services:** Redis object cache, vsftpd, Adminer, static site, cAdvisor.

github.com/zech-chi/1337-inception-42 ↗

ARCHITECTURE



04 — Systems

WebServ

Team project · HTTP/1.1 web server from scratch

C++98 · Sockets · kqueue · HTTP/1.1 · CGI

THE PROBLEM

Understand what really happens between a browser and a server by writing an HTTP server with nothing but sockets — it must stay responsive under many concurrent clients and never crash.

WHAT I BUILT

A configurable C++ web server with an nginx-style config file: multiple server blocks and ports, uploads, directory listings, redirects, CGI and sessions.

ENGINEERING HIGHLIGHTS

- **Event loop:** non-blocking sockets driven by `kqueue` — one process, many clients.
- **HTTP:** GET / POST / DELETE, chunked request bodies, body-size limits, custom error pages.
- **Config:** virtual hosts, route-specific methods, redirects and upload directories.
- **CGI:** configured interpreters with a five-second timeout; cookie & file-based sessions.

github.com/zech-chi/1337-Webserv-42 ↗

ARCHITECTURE



05 — Systems

Minishell

Team project · a bash-like shell written in C

C · readline · fork / execve · pipes · signals

THE PROBLEM

Rebuild a real Unix shell: read a command line, understand its grammar — quotes, pipes, redirections, logical operators — and run it exactly the way bash would.

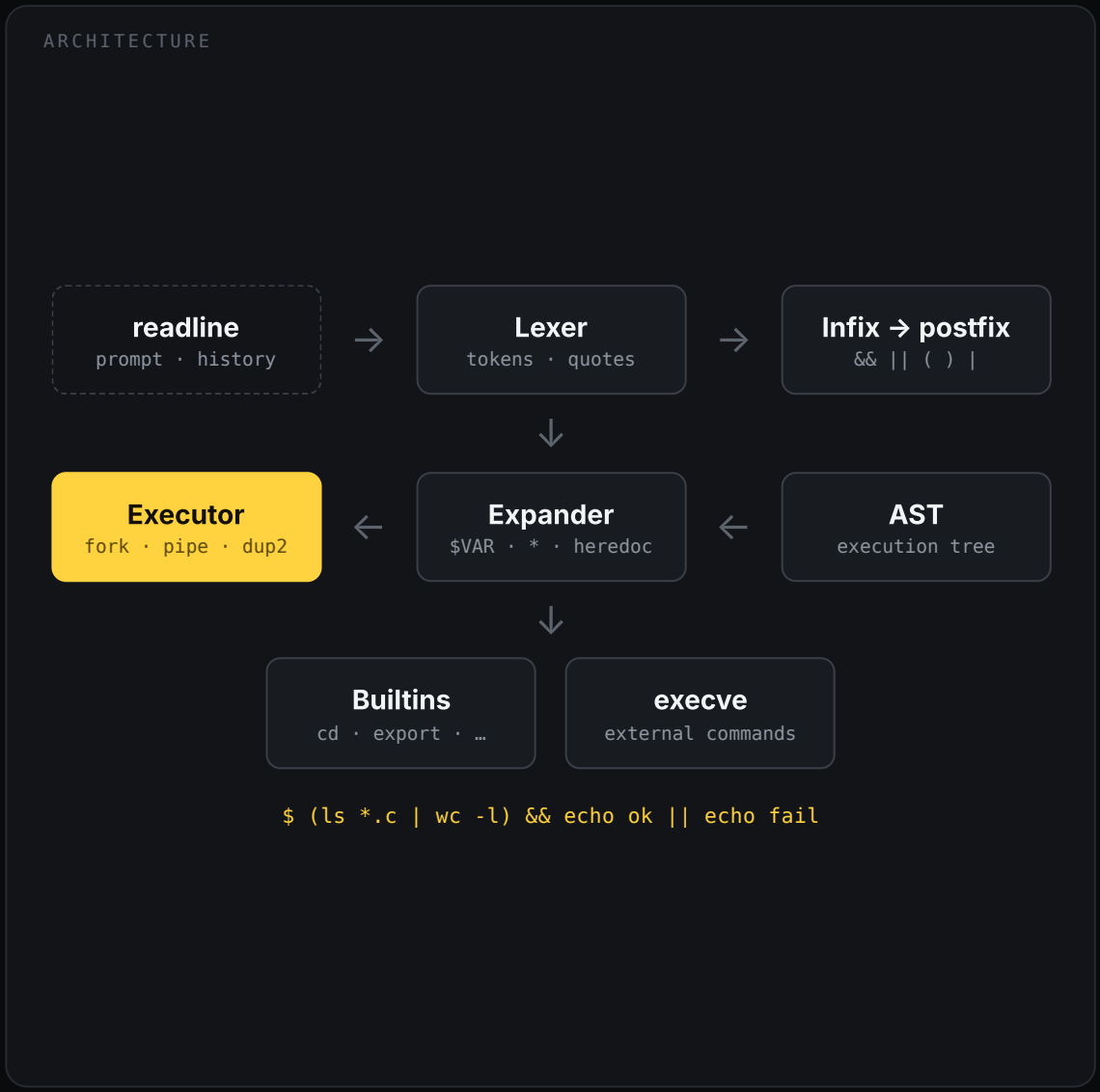
WHAT I BUILT

An interactive shell with its own lexer, parser and executor, including the bonus: `&&`, `||`, parentheses and wildcards.

ENGINEERING HIGHLIGHTS

- **Parsing:** tokens converted from infix to postfix, then built into an execution tree.
- **Execution:** `fork`, `execve`, `pipe` and `dup2` for pipelines and redirections.
- **Heredoc & expansion:** `<<` heredocs, `$VAR` / `$?` expansion and `*` wildcards.
- **Builtins:** cd, echo, env, exit, export, pwd, unset; Ctrl-C / Ctrl-\ handled like bash.

github.com/zech-chi/1337-Minishell-42 ↗



06 — Systems

Philosophers

Concurrency · the dining philosophers problem, twice

C · pthreads · mutexes · fork · POSIX semaphores

THE PROBLEM

N philosophers share N forks around a table. Each must eat before starving — without deadlocks, data races, or a death being reported a few milliseconds late.

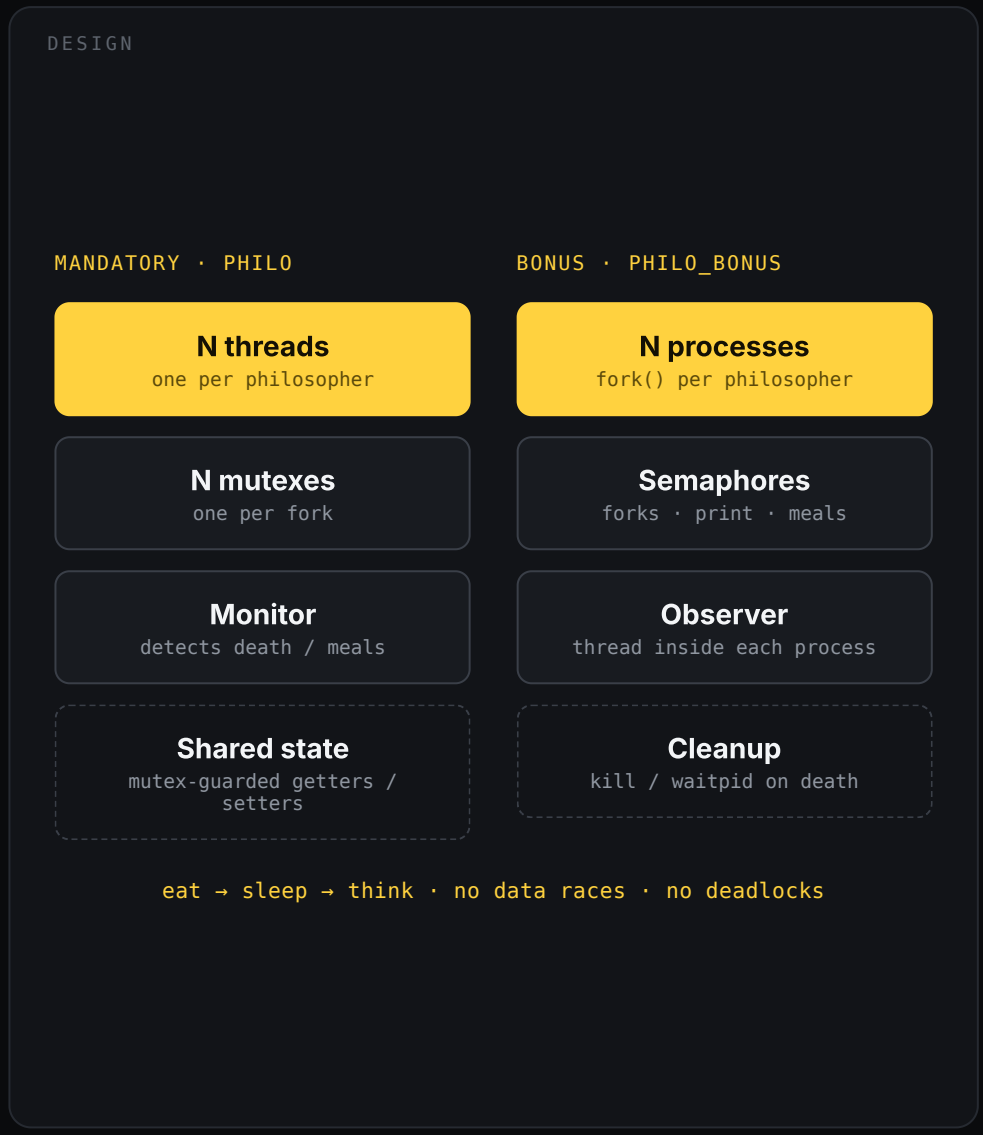
WHAT I BUILT

Two complete simulations: one with threads and mutexes, and a bonus version where each philosopher is its own process synchronised with named semaphores.

ENGINEERING HIGHLIGHTS

- **Threads:** each philosopher runs in a pthread; forks are mutexes.
- **Race-free state:** every shared value read and written through mutex-guarded helpers.
- **Processes:** bonus forks one process per philosopher, with semaphores for forks, printing and meal counts.
- **Timing:** millisecond-accurate sleeps and a monitor that reports death on time.

github.com/zech-chi/1337-Philosophers-42 ↗



07 — Security

Darkly

Web security · penetration testing of a vulnerable training site

Bash · curl · HTTP · OWASP

THE PROBLEM

A deliberately vulnerable web app. Find how small mistakes expose private data, hand over other people's accounts, or give control of the database — then prove it and explain the fix.

WHAT I BUILT

A security journal of 13 chapters: for each finding, how it was found, its impact, a reproducible demonstration, and how to prevent it.

ENGINEERING HIGHLIGHTS

- **Access control:** IDOR, mass assignment and forged JWTs leading to privilege escalation.
- **Injection & input:** stored XSS, path traversal, unrestricted uploads, SSRF.
- **Proof:** 11 Bash scripts that reproduce the attacks end-to-end.
- **Remediation:** a concrete fix for every issue, with its impact explained.

github.com/zech-chi/1337-Darkly-42 ↗

FINDINGS

01 Information disclosure	02 Insecure password reset
03 Broken access control (IDOR)	04 Unrestricted file upload
05 Stored XSS	06 Mass assignment → privilege escalation
07 JWT forgery (weak secret)	08 SSRF
09 Superuser access	10 LFI / path traversal
11 Open redirect	12 User enumeration
13 Security misconfiguration	

13	11	9
chapters	Bash scripts	flags

08 — Data

Analyze NFL games

Data analysis · what team stats are associated with winning?

Python · pandas · NumPy · Matplotlib · scikit-learn

THE PROBLEM

Which team performance measures actually go together with winning? Using 2021 NFL game statistics, explore the data and test it with a simple, interpretable model.

WHAT I BUILT

An analysis notebook that cleans and explores the stats, visualises the relationships, and trains a logistic regression classifier on completed game outcomes.

ENGINEERING HIGHLIGHTS

- **Data prep:** cleaning and structuring game-level statistics with pandas and NumPy.
- **Exploration:** Matplotlib visualisations of how each measure relates to wins.
- **Model:** logistic regression (scikit-learn), chosen for interpretability.
- **Interpretation:** reading the model to see which measures matter most.

github.com/zech-chi/Analyze-NFL-Stats-with-Python ↗



09 — Graphics

cub3D

Team project · first-person maze engine in C

C · MLX42 · Raycasting · pthreads

THE PROBLEM

Turn a 2D grid in a `.cub` file into an explorable first-person 3D world — in C, with a minimal graphics library and no engine.

WHAT I BUILT

A Wolfenstein-style raycaster: the mandatory build is the core renderer; the bonus adds a minimap, mouse look, weapon animations, sound and more.

ENGINEERING HIGHLIGHTS

- **Raycasting:** 1,200 rays across a 60° field of view, projecting textured wall columns.
- **Gameplay:** movement, turning, wall collision; doors and target tiles in bonus.
- **Bonus:** circular minimap, sky, mouse & vertical look, jump, head bobbing, shoot / reload with ammo.
- **Audio & parsing:** music and action sounds on their own thread; strict `.cub` validation.

github.com/zech-chi/1337-cub3d-42 ↗

SCREENSHOTS



bonus · minimap · sky · weapon



reload animation



textured walls



mandatory build

10 — Graphics

so_long

2D top-down game in C

C · MiniLibX · BFS · XPM sprites

THE PROBLEM

A first graphics project: read a map file, make sure it is actually winnable, and turn it into a small game — collect every coin, then reach the exit.

WHAT I BUILT

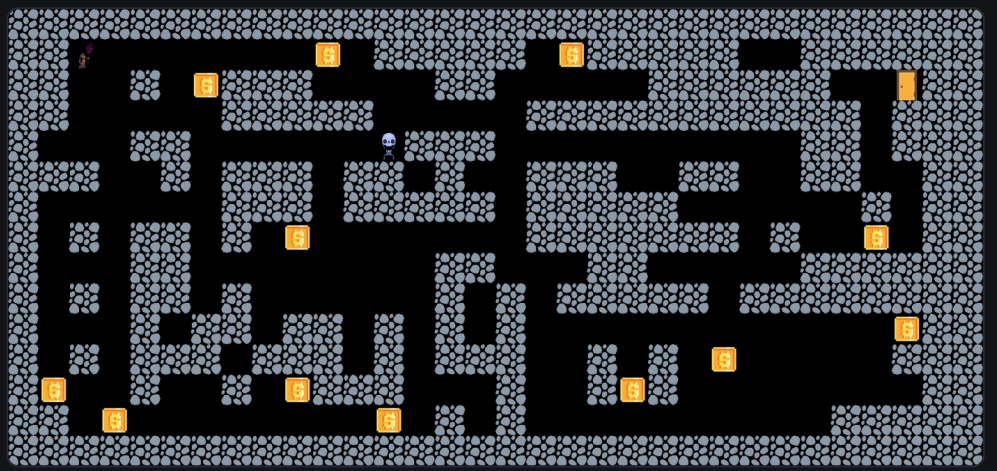
A tile-based game with mandatory and bonus builds; the bonus adds patrolling enemies, sprite animations and an on-screen move counter.

ENGINEERING HIGHLIGHTS

- **Map validation:** rectangular, closed by walls, one player / exit, at least one coin.
- **Path check:** breadth-first search proves every coin and the exit are reachable.
- **Rendering:** MiniLibX window drawing XPM tiles; map read with get_next_line.
- **Bonus:** animated coins and enemies, enemy collision = game over, moves shown on screen.

github.com/zech-chi/1337-so-Long-42 ↗

IN GAME



a bonus map rendered from the game's own tiles



enemy · 8 frames



coin · 9 frames

$ contact --now

Let's build something.

Open to roles and projects in **AI engineering**, **software engineering** and **security**. All source code is on GitHub.

EMAIL
zech.chi.work@gmail.com

GITHUB
github.com/zech-chi

LINKEDIN
in/zakaria-ech-chifaouy

PHONE
+212 601 411 639